시스템프로그래밍

Assignment1-2

제출일자: 2026-04-04

담당 교수님: 최상호

학번: 2022202094

이름: 이원우

분반: 금 1,2 (B)

Introduction

본 과제는 Linux 환경에서 디렉토리 정보를 다루는 시스템 프로그래밍 기초를 이해하기 위해 진행되었다.
특히 opendir(), readdir(), closedir() 함수를 활용하여 디렉토리 내부의 파일 목록을 출력하는 프로그램을 구현하는 것을 목표로 한다.
또한 단순 출력뿐만 아니라, 잘못된 입력이나 접근 권한 문제 등 다양한 예외 상황을 처리함으로써 실제 시스템 환경에서의 안정적인 프로그램 동작을 경험하는 데 목적이 있다.
이를 통해 파일 시스템 접근 방식과 에러 처리 방식에 대한 이해를 높이고자 한다.

결과 화면

현재 디렉토리

```
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ ./kw2022202094_ls
..
kw2022202094_ls.c
Makefile
t1
.
kw2022202094_ls
```

정상 디렉토리

```
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ ./kw2022202094_ls /dev
.
..
loop11
vcsa6
vcsu6
vcs6
vcsa5
vcsu5
vcs5
vcsa4
vcsu4
vcs4
vcsa3
vcsu3
vcs3
vcsa2
vcsu2
vcs2
vsock
dmmidi
midi
dvd
cdrw
cdrom
vmci
loop10
loop9
loop8
dri
fb0
zfs
snd
vhost-vsock
vhost-net
uhid
vhci
userio
nvram
btrfs-control
cuse
cpu
mqueue
hugepages
log
initctl
shm
autofs
disk
sda5
sda2
sda1
sda
sg1
bsg
sg0
sr0
hidraw0
block
rtc
char
stderr
stdout
stdin
fd
core
pts
cpu_dma_latency
mcelog
mapper
rtc0
uinput
psaux
input
bus
vfio
ppp
net
udmabuf
dma_heap
loop7
loop6
loop5
loop4
loop3
loop2
loop1
loop0
loop-control
hwrng
hpet
ttyprintk
ttyS31
ttyS30
ttyS29
ttyS28
ttyS27
ttyS26
ttyS25
ttyS24
ttyS23
ttyS22
ttyS21
ttyS20
ttyS19
ttyS18
ttyS17
ttyS16
```

```
ttyS15
ttyS14
ttyS13
ttyS12
ttyS11
ttyS10
ttyS9
ttyS8
ttyS7
ttyS6
ttyS5
ttyS4
ttyS3
ttyS2
ttyS1
ttyS0
ptmx
fuse
ecryptfs
snapshot
tty63
tty62
tty61
tty60
tty59
tty58
tty57
tty56
tty55
tty54
tty53
tty52
tty51
tty50
tty49
tty48
tty47
tty46
tty45
tty44
tty43
tty42
tty41
tty40
tty39
tty38
tty37
tty36
tty35
tty34
tty33
tty32
tty31
tty30
tty29
tty28
tty27
tty26
tty25
tty24
tty23
tty22
tty21
tty20
tty19
tty18
tty17
tty16
tty15
tty14
tty13
tty12
tty11
tty10
tty9
tty8
tty7
tty6
tty5
tty4
tty3
tty2
tty1
vcsa1
vcsu1
vcs1
vcsa
vcsu
vcs
tty0
console
tty
kmsg
urandom
random
full
zero
port
null
mem
rfkill
vga_arbiter
```

존재하지 않는 디렉토리

```
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ ./kw2022202094_ls /not_
exist_dir
./kw2022202094_ls: cannot access '/not_exist_dir' : No such directory
```

권한 없는 디렉토리

```
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ mkdir testdir
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ chmod 000 testdir
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ ./kw2022202094_ls testd
ir
./kw2022202094_ls: cannot access 'testdir' : Access denied
```

일반 파일 입력

```
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ touch t1
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ ./kw2022202094_ls t1
./kw2022202094_ls: cannot access 't1' : No such directory
```

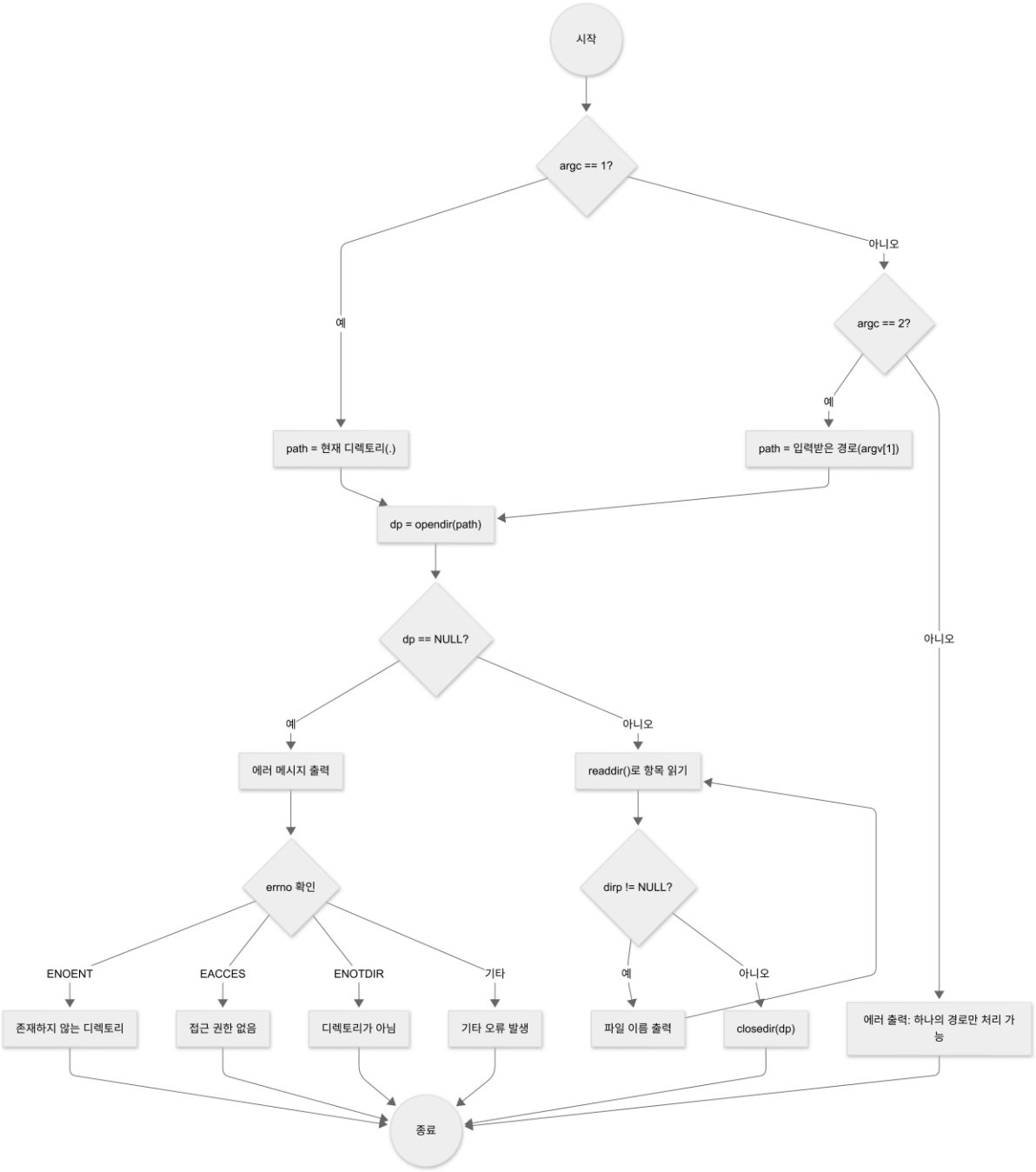
인자 여러 개

```
kw2022202094@ubuntu:~/Assignment1_2_B_2022202094_이원우$ ./kw2022202094_ls ./ ..
/
only one directory path can be processed
```

프로그램은 크게 입력 처리, 디렉토리 접근, 파일 출력, 예외 처리의 흐름으로 구성된다.
먼저 main 함수에서 전달되는 인자(argc, argv)를 기반으로 입력 경로를 결정하였다.
인자가 없는 경우에는 현재 디렉토리(".")를 기본 경로로 설정하였고, 인자가 하나일 경우 해당 경로를 사용하도록 하였다.
두 개 이상의 인자가 전달될 경우에는 과제 요구사항에 따라 에러 메시지를 출력하고 프로그램을 종료하도록 구현하였다.
이후 opendir() 함수를 사용하여 지정된 경로의 디렉토리를 열었다.
디렉토리 열기에 실패할 경우 errno 값을 활용하여 실패 원인을 구분하였다.
존재하지 않는 경로(ENOENT), 접근 권한 문제(EACCES), 디렉토리가 아닌 경우(ENOTDIR)를 각각 처리하도록 분기하였다.
디렉토리가 정상적으로 열릴 경우 readdir() 함수를 반복 호출하여 디렉토리 내부의 항목을 하나씩 읽어들였다.
각 항목의 이름은 dirent 구조체의 d_name 을 통해 얻을 수 있으며, 이를 출력하였다.
모든 항목을 출력한 후에는 closedir()을 호출하여 디렉토리 스트림을 정상적으로 종료하였다.

Algorithm

Flow Chart



Pseudo Code

```
START



initialize path = NULL

IF (argc == 1)
    set path = "."
ELSE IF (argc == 2)
    set path = argv[1]
ELSE
    print "only one directory
path can be processed"
    terminate program
END IF


dp = opendir(path)

IF (dp == NULL)
    print "program_name:
cannot access 'path' : "

    IF (error is ENOENT)
        print "No such directory"
    ELSE IF (error is EACCES)
        print "Access denied"
    ELSE IF (error is ENOTDIR)
        print "No such directory"
    ELSE
        print "operation failed"
    END IF

    terminate program
END IF


WHILE (readdir(dp) is not
NULL)
    read one directory entry
    print entry name
END WHILE


close directory

terminate program successfully
END
```

고찰

본 과제를 통해 디렉토리 처리 함수의 기본적인 사용법과 시스템 콜의 동작 방식을 이해할 수 있었다. 특히 readdir()를 이용해 디렉토리 내부를 순차적으로 탐색하는 구조가 인상적이었다. 단순한 기능 구현보다 예외 처리가 더 중요하다는 점도 확인할 수 있었다. opendir() 호출이 실패했을 때 단순히 프로그램을 종료하는 것이 아니라, errno 라는 헤더를 활용해 원인을 구분함으로써 사용자에게 보다 정확한 정보를 제공할 수 있었다.
과제를 수행하는 과정에서 Linux 환경의 파일 권한 개념과 디렉토리 접근 제한에 대해서도 이해하게 되었으며, 시스템 프로그래밍이 단순한 코드 작성이 아니라 운영체제와의 상호작용이라는 점을 체감할 수 있었다.